

Un agent, ses outils

Ce qu'un outil expose, ce qu'il rend, et ce qu'il ouvre. Conception, exécution, sécurité.

Denis Lamard

Concepteur et intégrateur d'agents IA

github.com/denislamard

Version 1.4, 2026

Sommaire

1. Il accède à quoi, exactement ?
2. Le vocabulaire, et ce qu'il recouvre vraiment
3. Le trajet d'un appel d'outil
4. Les 4 natures d'outil
5. Concevoir un outil : le découpage
6. La description est le routage
7. Ce que l'outil rend au modèle
8. L'exécution
9. Effets de bord, état et reprise
10. Sécurité : la carte, puis le document
11. Tester et évaluer un catalogue d'outils
12. Observer et chiffrer
13. Faire vivre le catalogue
14. Ce que je retiens
15. À propos

Annexe A. Lexique

Annexe B. Fiche de mise en production d'un outil

Annexe C. Anatomie d'un outil, en code

Annexe D. Relevé daté

1. Le modèle demande, votre code exécute

« Et l'agent IA accède à quoi, exactement ? »

La question arrive rapidement, une fois le principe acquis, et elle vient presque toujours du développeur qui devra vivre avec le système. Et c'est la bonne question. Ce qu'un agent est autorisé à faire tient dans une liste, cette liste s'appelle un catalogue d'outils, et elle est écrite par une personne identifiable.

Un modèle de langage produit du texte. Seul, il ne lit aucun fichier, n'appelle aucune API et ne modifie aucune base. Il ignore même la date du jour : MiniMax M3 répond à côté dès qu'une question porte sur « depuis combien de temps ». Savoir la date et l'heure demande déjà un outil à ce modèle. Ce qu'il sait faire de plus, c'est demander : émettre un nom de fonction et des arguments, sous une forme que votre code sait reconnaître. Vérifier que la demande est recevable, l'autoriser, l'exécuter, la borner, et rendre un résultat lisible : tout cela est du code que vous écrivez. C'est le sujet de ce document.

Deux conséquences, une pour chaque lecteur. Pour celui qui achète : le périmètre d'action d'un agent est fermé par construction et il se lit dans un fichier de configuration. Pour celui qui construit : la plus grande partie du travail de conception d'un agent est un travail d'interface. Le prompt système occupe une page, le catalogue d'outils occupe des semaines, et c'est lui qui décide de la tenue du système en production.

Ce guide est écrit pour un développeur. Chaque section s'ouvre sur ce qui se passe, en langue courante, avant de descendre dans le détail de mise en œuvre. Un lecteur non technique lit les ouvertures, les tableaux et les encadrés, et suit l'argument de bout en bout. Le code est rassemblé en annexe C.

Le cas qui sert de fil rouge est composé à partir de 3 agents que j'exploite ou que j'ai exploités : un agent de relance de devis en service, un agent vocal, et un assistant documentaire. Les outils décrits existent, leur assemblage dans un même agent est une commodité de lecture. Les chiffres relevés en production sont attribués à l'agent qui les a produits, jamais au cas composite.

Le fil rouge : un agent d'accueil

Une entreprise de travaux d'une dizaine de compagnons reçoit des demandes par formulaire, par courriel et par téléphone. L'agent d'accueil les prend en charge : il retrouve le demandeur s'il est déjà client, lit son dossier, comprend la demande à partir du texte et des photos jointes, cherche un créneau de visite, prépare un devis en brouillon, et relance ce qui reste sans réponse.

Son catalogue tient en 8 outils. Ce sont les mêmes 8 outils d'un bout à l'autre du document, examinés dans chaque partie sous l'angle qu'elle traite.

Outil	Ce qu'il fait	Nature
chercher_client	Retrouve un client à partir d'un nom, d'un téléphone ou d'un courriel	Lecture locale
lire_dossier	Rend l'historique d'un client : devis, chantiers, factures, échanges	Lecture locale
chercher_creneaux	Interroge l'agenda de l'entreprise sur une plage donnée	Service distant
analyser_photo	Décrit une photo de chantier et en extrait les éléments techniques	Modèle spécialisé
chercher_documentation	Cherche dans les fiches produit et les documents techniques	Recherche documentaire
creer_devis_brouillon	Crée un devis à l'état brouillon dans le logiciel de facturation	Écriture réversible
poser_rendez_vous	Pose une visite dans l'agenda et l'annonce au client	Écriture irréversible
envoyer_message	Envoie un courriel ou un SMS au client	Écriture irréversible

2. Le vocabulaire, et ce qu'il recouvre vraiment

Les mots de ce domaine changent de sens selon qui les prononce, et plusieurs décisions d'architecture se prennent sur des malentendus de définition. Cette partie pose les termes. Un lecteur déjà au fait peut la parcourir et reprendre à la partie 3.

2.1 Ce qu'est un outil

Un outil se compose de 4 éléments. Le modèle en voit 3.

- **Un nom.** Un identifiant stable, que le modèle écrira dans sa demande.
- **Une description.** Du texte libre qui dit ce que l'outil fait et quand l'appeler. C'est du prompt, et c'est la partie qui décide du routage.
- **Un schéma d'entrée.** La liste des arguments attendus, leur type et leur caractère obligatoire, exprimée en JSON Schema.
- **Une exécution.** La fonction qui fait le travail. Elle reste chez vous, le modèle ne la voit jamais et n'en connaît que le résultat.

Ce résultat revient dans la conversation sous forme de texte, à la suite de la demande qui l'a produit. Le modèle lit ce texte comme il lirait n'importe quel autre message, et il en tire sa décision suivante. Retenez cette phrase, la partie 10 s'appuie entièrement dessus.

2.2 Function calling, tool use, tool calling

Trois expressions pour un seul mécanisme. Le fournisseur reçoit votre liste d'outils avec la conversation, et le modèle répond soit par du texte, soit par une ou plusieurs demandes d'appel structurées. Deux formats dominent le paysage, celui d'Anthropic et celui d'OpenAI, et la plupart des autres fournisseurs exposent l'un des deux. Un socle d'orchestration correctement découpé absorbe cet écart dans sa couche fournisseur, ce qui rend le reste du système indifférent au format.

Un paramètre mérite d'être connu, souvent nommé **tool_choice**. Il prend trois valeurs utiles : laisser le modèle décider, lui interdire les outils pour forcer une réponse rédigée, ou imposer un outil précis. La deuxième valeur porte la sortie de secours d'un agent, décrite en partie 8.

2.3 Qui décide quoi

La répartition est nette, et elle vaut d'être montrée telle quelle à un directeur technique inquiet.

Sur quoi	Le modèle	Le code
Le catalogue	choisit dans la liste qu'il reçoit	décide de la liste, et de ce qui est visible à ce tour
Les arguments	les propose	les valide contre le schéma
L'appel	le demande	l'autorise, et choisit sous quelle identité
L'exécution	l'ignore	la lance, la borne dans le temps, la journalise
Le résultat	l'interprète	décide de ce qu'il contient
La fin	s'arrête quand il a répondu	impose un nombre de tours et un plafond de dépense

La ligne qui compte pour la sécurité : le modèle formule une intention, le code garde la main sur l'effet. Pour qu'un agent écrive à un client, il faut qu'un développeur ait écrit l'outil *envoyer_message*, l'ait déclaré dans le catalogue, et lui ait ouvert un accès à la messagerie. Trois gestes délibérés, tracés dans un dépôt.

2.4 MCP, en une page

Le **Model Context Protocol** est un protocole ouvert qui normalise la façon dont un programme expose des outils à un agent. Un serveur MCP déclare ce qu'il sait faire, l'agent le découvre au démarrage, et les appels passent ensuite par un canal standard : entrée et sortie standard pour un serveur local STDIO, HTTP pour un serveur distant.

Ce qu'il apporte : un outil écrit une fois sert plusieurs agents, y compris des agents que vous n'écrivez pas. GitHub publie ainsi un serveur MCP officiel, que Claude Code et les autres agents de code utilisent pour lire un dépôt et ouvrir une pull request. Un éditeur peut livrer l'accès à son logiciel sous cette forme, et votre agent le consomme sans que vous ayez à connaître son API.

La contrepartie, quand le serveur est tenu par un tiers : le code que votre agent exécute vit dans le dépôt de quelqu'un d'autre. Sa version change quand son éditeur le décide, sans passer par votre mise en production. Et sa description entre dans le prompt de votre agent, donc un texte que vous n'avez pas écrit oriente les décisions du modèle. La partie 10 revient sur ce dernier point, qui est le moins visible des trois.

Un serveur MCP que vous écrivez vous-même échappe à ces trois points. Il sert alors à mutualiser vos outils entre plusieurs agents, ou à isoler une capacité dans son propre processus, et vous gardez la main sur tout.

2.5 Ce qu'un outil n'est pas

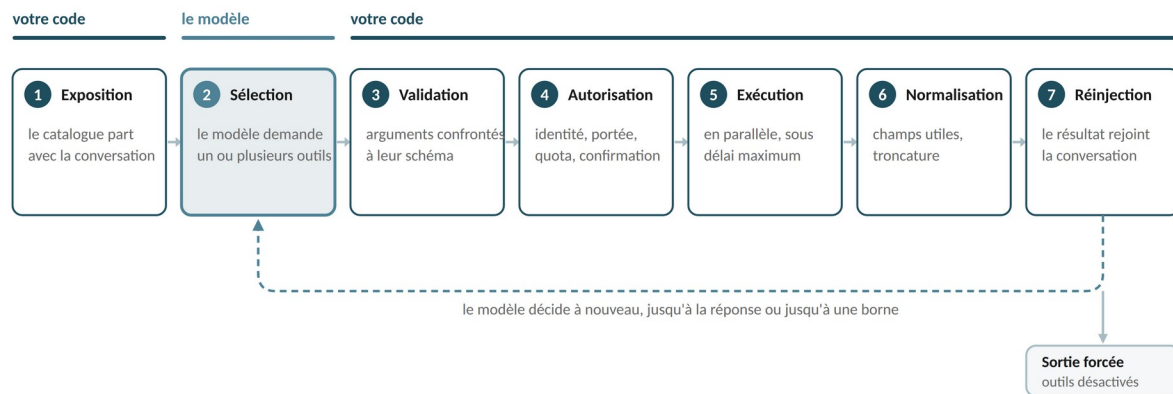
Un outil n'est ni un plugin propriétaire ni une intégration no-code : c'est une fonction de votre code, déclarée avec un contrat.

La recherche documentaire, souvent appelée **RAG**, est un outil parmi les autres. Le débat entre agent et RAG se dissout dès qu'on le regarde ainsi : la recherche devient un outil que le modèle appelle quand il en a besoin, avec les mots-clés qu'il choisit, éventuellement plusieurs fois en affinant sa requête au vu des premiers résultats.

Un agent possède une boucle, une mémoire et un catalogue. Un outil rend un résultat et rend la main. La partie 4 montre le cas intermédiaire, un modèle spécialisé exposé comme outil, qui emprunte aux deux.

3. Le trajet d'un appel d'outil

Voici ce qui se passe entre le moment où un client écrit « fuite sous l'évier, photo jointe » et le moment où l'agent propose un créneau. La séquence est toujours la même, quelle que soit la demande, et elle sert de carte au reste du document : chaque partie suivante travaille une de ses 7 étapes.



1. Exposition. À chaque appel au modèle, le catalogue est réexpédié avec la conversation : pour chaque outil, son nom, sa description et son schéma. C'est du contexte facturé, à chaque tour, et c'est la raison pour laquelle un catalogue se conçoit avec la même économie qu'un prompt système.

2. Sélection. Le modèle lit la demande et le catalogue, et répond par une ou plusieurs demandes d'appel. Sur notre exemple, il en émet deux d'un coup : **chercher_client** sur le numéro de téléphone, et **analyser_photo** sur la pièce jointe. Rien n'oblige un modèle à n'en demander qu'un.

3. Validation des arguments. Chaque demande est confrontée à son schéma avant toute exécution. Un argument manquant, un type incorrect, une valeur hors énumération : l'appel est refusé à ce stade et le motif du refus est rendu au modèle, qui corrige au tour suivant. Un modèle se trompe d'arguments régulièrement, et c'est une situation normale de fonctionnement.

4. Autorisation. Un appel valide peut rester interdit. C'est ici que se placent les décisions qui ne regardent pas le modèle : l'identité sous laquelle l'outil s'exécute, la portée des données accessibles, la confirmation humaine exigée pour un effet irréversible, le quota déjà consommé.

5. Exécution bornée. L'appel part, avec un délai maximum et une politique de reprise. Les appels du même tour partent ensemble. Une exception levée par un outil devient un résultat d'erreur, sans interrompre la boucle.

6. Normalisation du résultat. Le retour est transformé en texte utile : champs retenus, unités explicites, troncature au-delà d'un seuil. C'est l'étape la plus négligée et l'une des plus rentables, la partie 7 lui est consacrée.

7. Réinjection et nouvelle décision. Les résultats rejoignent la conversation, le modèle relit l'ensemble et recommence à l'étape 2. La boucle tourne jusqu'à ce qu'il réponde, ou jusqu'à ce qu'une borne l'arrête.

Sur notre exemple, le deuxième tour appelle **lire_dossier** avec l'identifiant retrouvé, le troisième appelle **chercher_creneaux** sur les quinze jours à venir, et le quatrième rédige la proposition. Quatre tours, six appels d'outils, une réponse.

4. Les 4 natures d'outil

Le modèle voit un nom, une description et un schéma. Derrière cette forme unique, 4 choses très différentes peuvent s'exécuter, et le choix de la nature engage la latence, le coût et le périmètre de confiance.

La fonction locale. Du code Python dans votre dépôt : une requête préparée, un appel à votre API interne, un calcul. C'est la nature par défaut, la plus rapide, la plus testable et celle sur laquelle vous avez tous les leviers. **chercher_client** et **lire_dossier** sont de celles-là.

Le service distant. Un serveur MCP, tenu par vous ou par un tiers, découvert au démarrage. **chercher_creneaux** interroge l'agenda de l'entreprise par ce canal. L'intérêt est la réutilisation, le prix à payer est un aller-retour réseau et une dépendance dont vous ne maîtrisez ni la version ni la disponibilité.

Le modèle spécialisé. Un second appel au modèle, avec son propre prompt système, exposé à l'orchestrateur sous forme d'outil. **analyser_photo** est un rôle de vision : il reçoit l'image, rend une description structurée, et l'orchestrateur travaille sur ce texte. C'est ce mécanisme qui rend un système multi-agents accessible sans framework multi-agents, et le guide précédent de cette série en traite le versant modèle et coût.

La recherche documentaire. Une recherche vectorielle, lexicale ou mixte sur un corpus, exposée comme un outil dont l'argument est une requête. **chercher_documentation** en est une. Sa particularité tient à ce qu'elle rend : du texte rédigé par d'autres, sur lequel la partie 10 demande de la vigilance.

Nature	Latence typique	Coût	Périmètre de confiance
Fonction locale	quelques millisecondes	négligeable	votre dépôt
Service distant	dizaines à centaines de ms	négligeable	l'éditeur du serveur
Modèle spécialisé	1 à plusieurs secondes	un appel de modèle	votre configuration
Recherche documentaire	dizaines de ms	l'indexation, puis le contexte rendu	les auteurs du corpus

Une remarque de conception plutôt que de technique : ces 4 natures se remplacent l'une l'autre sans que l'orchestrateur le sache. **analyser_photo** peut commencer en modèle

spécialisé, puis devenir un appel à un service interne le jour où votre entreprise en achète un. Le catalogue reste identique, la configuration change. C'est ce que vous achetez en tenant l'interface propre.

5. Concevoir un outil : le découpage

Le premier réflexe, quand on branche un agent sur un système existant, consiste à exposer ce système tel quel. Deux outils suffisent alors : un pour interroger la base, un pour appeler n'importe quelle URL. Ils marchent en démonstration. Ils expliquent aussi la plupart des agents qui déraillent en production.

5.1 Un verbe métier plutôt qu'un miroir d'API

Comparons deux façons de donner accès aux mêmes données.

Version générique	Version métier
<code>requete_sql(requete)</code>	<code>chercher_devis_impayees(client, depuis)</code>
<code>appel_api(url, methode, corps)</code>	<code>creer_devis_brouillon(client, lignes)</code>

La version générique demande au modèle d'écrire du SQL correct sur un schéma qu'il ne connaît pas, elle rend un résultat dont la forme change à chaque appel, elle ouvre l'accès à toutes les tables, et elle rend le test impossible puisque l'espace des appels possibles est infini. La version métier fixe l'intention, contraint les arguments, rend une forme stable, et se teste avec dix cas.

Le repère : **un outil correspond à une action que vous sauriez nommer dans une réunion avec le client**. Si le nom de l'outil contient un mot de votre couche technique, il y a probablement un cran de découpage à remonter.

5.2 Combien d'outils

La qualité de sélection se dégrade quand le catalogue grossit, pour une raison mécanique : le modèle choisit dans une liste, et plus la liste contient d'entrées proches, plus le choix devient une comparaison fine. Sur mes agents, entre 5 et 15 outils constitue une zone confortable, et au-delà de 20 la sélection mérite d'être mesurée avant d'être crue.

Trois leviers, quand le catalogue enfle. Fusionner deux outils voisins en un seul avec un argument d'énumération. Masquer les outils hors sujet au tour courant, ce que la partie 6 détaille. Déléguer un sous-domaine entier à un modèle spécialisé qui porte son propre catalogue, ce qui ramène 6 outils à 1 du point de vue de l'orchestrateur.

5.3 Séparer la lecture de l'écriture

Un outil qui lit et un outil qui écrit ne relèvent pas du même régime, et les mélanger dans une même fonction coûte cher plus tard. La séparation permet d'exécuter les lectures sous une identité restreinte, d'autoriser les écritures une par une, de rejouer une conversation entière sans effet de bord, et de tester le raisonnement de l'agent sans toucher à la production.

Sur le fil rouge, la coupure est visible : 5 outils lisent, 3 écrivent. Les 3 outils d'écriture concentrent la totalité des questions de sécurité, de reprise et de confirmation. Les 5 autres concentrent les questions de coût et de contexte.

5.4 Les arguments

Un schéma d'entrée est un contrat, et un contrat court se respecte mieux qu'un contrat long.

- **Peu d'arguments.** Au-delà de 5, le taux d'appels mal formés monte. Un argument que vous pouvez déduire côté code n'a rien à faire dans le schéma.
- **Des types fermés.** Une énumération à 4 valeurs vaut mieux qu'une chaîne libre. Le modèle choisit dans une liste au lieu d'inventer un intitulé.
- **Des unités dans le nom.** `delai_jours` se trompe moins que `delai`.
- **Des valeurs par défaut côté code.** Elles réduisent le schéma et retirent une occasion d'erreur.
- **Une description par argument.** Le schéma JSON accepte un champ de description sur chaque propriété, et il est lu par le modèle. C'est l'endroit où préciser un format de date ou une convention d'identifiant.

5.5 L'idempotence

Un agent réessaie. Le réseau tombe, un timeout se déclenche, un processus redémarre, et le même appel repart. Si cet appel envoie un message, le client reçoit deux messages.

La réponse est connue depuis longtemps et elle vient d'un autre métier : une **clé d'idempotence**. L'appelant fournit un identifiant qui caractérise l'intention, le service enregistre cette clé avec son résultat, et un second appel portant la même clé rend le résultat du premier sans rien réexécuter. Sur **envoyer_message**, la clé peut être construite à partir de l'identifiant du dossier, du type de message et du jour.

J'ai passé 7 ans sur des migrations de données bancaires, où un traitement doit s'exécuter une fois et une seule, reprendre après incident et se laisser auditer. La spécification d'un agent en production est la même, posée vingt ans plus tôt.

Un outil qui écrit sans clé d'idempotence est un incident en attente de son premier timeout. C'est la seule règle de cette partie qui ne souffre aucune exception.

6. La description est le routage

Dans un agent, le routage ne vit pas dans le code. Aucune instruction **if** ne dit quand appeler **chercher_documentation** plutôt que **lire_dossier** : c'est la description de chaque outil qui le dit, et le modèle arbitre à partir de ces phrases. Écrire une description est donc un acte de programmation, avec la particularité que le compilateur est un modèle de langage.

6.1 Ce qu'une description doit contenir

Quatre éléments, dans cet ordre.

- **Ce que l'outil fait**, en une phrase, au présent.
- **Quand l'appeler**, avec une situation reconnaissable dans la conversation.
- **Ce qu'il rend**, pour que le modèle sache ce qu'il pourra en faire.
- **Ce qui le distingue de son voisin le plus proche**, quand il y en a un.

Le quatrième point est celui qui manque presque toujours, et c'est celui qui produit les erreurs de routage les plus coûteuses. Sur le fil rouge, **chercher_documentation** et **lire_dossier** répondent tous deux à « je cherche une information ». Une phrase suffit à les séparer : l'un porte l'historique de ce client, l'autre porte les fiches techniques des produits, communes à tous les clients.

6.2 Ce qu'elle coûte

Le catalogue est réexpédié à chaque tour. Une description de 80 mots, sur un catalogue de 8 outils, dans une conversation de 6 tours, se paie 6 fois. Sur les modèles frontier, la mise en cache de la partie fixe du prompt ramène ce coût à une fraction, et le catalogue est justement une partie fixe : il gagne à être placé avec le prompt système, dans la zone mise en cache, plutôt que reconstruit à chaque appel.

Le surcoût du français mérite d'être rappelé ici. Un même contenu consomme davantage de tokens en français qu'en anglais, avec un écart courant de 15 à 30 %. Rédiger les descriptions en anglais et les messages destinés au client en français est un arbitrage défendable, à condition de le tenir sur tout le catalogue.

6.3 Le masquage conditionnel

Tous les outils n'ont pas à être visibles à tous les tours. Retirer de la liste un outil hors sujet évite au modèle un appel inutile et un tour complet.

L'exemple vient de mon socle : quand un tour ne porte aucune image, les rôles de vision sont retirés du catalogue. Sans ce filtre, l'agent appelle le rôle, récupère « appelé sans image », puis refait un tour pour demander une photo. Deux itérations et six secondes pour rien. Le même raisonnement s'applique aux outils d'écriture tant qu'une étape de lecture n'a pas eu lieu, et aux outils d'un domaine que la conversation n'a pas encore abordé.

6.4 Écrire, mesurer, réécrire

Une description se corrige comme un prompt : en observant les appels réels. Les trois symptômes qui demandent une réécriture sont l'outil jamais appelé alors qu'il devrait l'être, l'outil appelé à la place d'un autre, et l'outil appelé deux fois de suite avec des arguments proches. La partie 11 donne la mesure qui rend ces symptômes visibles.

Réécrire la description d'un outil coûte quelques minutes et se mesure le jour même. Changer de modèle pour corriger un problème de routage coûte une migration, et laisse le problème intact.

7. Ce que l'outil rend au modèle

Un outil qui rend 40 000 caractères ne coûte rien à l'exécution. Il coûte à chaque tour suivant, puisque son résultat reste dans la conversation, et il coûte en qualité, puisque l'information utile s'y noie. C'est la partie du travail où un développeur habitué aux API doit changer de réflexe : le destinataire n'est pas un programme qui sait ignorer les champs inutiles, c'est un lecteur qui paie chaque mot.

7.1 Rendre peu, et rendre net

4 règles tiennent la plus grande part du résultat.

- **Les champs utiles à la décision, et eux seuls.** `lire_dossier` rend le nombre de chantiers, les 3 derniers avec leur date et leur montant, et l'état de paiement. Les identifiants techniques, les horodatages de création et les champs de traçabilité restent côté code.
- **Une forme stable.** Le même outil rend toujours la même structure, y compris quand il ne trouve rien. Un résultat vide se dit explicitement.
- **Des unités et des dates absolues.** « il y a 3 jours » se recalcule mal au tour suivant, « le 14 mars 2026 » se réutilise tel quel.
- **Une troncature explicite, avec sa suite.** Au-delà d'un seuil, le résultat est coupé et le modèle est informé qu'il l'a été, avec le moyen d'obtenir la suite. Mon socle tronque à 50 000 caractères et le dit dans le corps du résultat. Un outil qui peut rendre beaucoup gagne à porter une pagination dès sa conception.

7.2 Une erreur est un résultat

Le réflexe naturel consiste à laisser remonter l'exception. Dans une boucle d'agent, cela arrête tout, pour une situation qui est souvent rattrapable au tour suivant.

Le bon comportement : capturer, et rendre au modèle un message d'erreur écrit pour lui. Un message d'erreur utile dit ce qui a échoué, pourquoi, et ce qui est possible ensuite.

Message inutile	Message actionnable
KeyError: 'client_id'	Aucun client ne correspond à ce numéro. Essayez avec le nom de famille, ou proposez la création d'une fiche.
HTTP 429	L'agenda est momentanément saturé. Ce même appel sera possible dans quelques secondes.
ValidationError	Le champ <code>delai_jours</code> attend un entier entre 1 et 90. Valeur reçue : « deux semaines ».

Sur mes agents, la troisième forme se corrige au tour suivant dans la quasi-totalité des cas, sans intervention. Une distinction reste nécessaire : une erreur métier est rendue au modèle, une erreur d'infrastructure qui rendrait toute suite absurde arrête l'exécution proprement et laisse une trace. Le premier cas fait partie du fonctionnement, le second fait partie de l'exploitation.

8. L'exécution

Cette partie s'adresse à l'équipe qui écrit le code. Elle traite de ce qui se passe entre la décision d'appeler un outil et le retour de son résultat, et de la façon dont on tient cette phase quand plusieurs appels partent ensemble.

8.1 Le parallélisme

Un modèle peut demander plusieurs outils dans le même tour, et les exécuter en série revient à additionner leurs latences. Sur le fil rouge, **chercher_client** et **analyser_photo** partent ensemble : l'un rend en quelques millisecondes, l'autre en une à deux secondes, et le tour coûte le temps du plus lent.

En Python, ces appels se lancent dans un **asyncio.gather**. Une contrainte en découle : un outil synchrone qui bloque la boucle prive tous les autres de leur parallélisme. Une fonction bloquante se déporte dans un fil d'exécution dédié, ce qui rend la question du parallélisme indépendante de la façon dont chaque outil est écrit.

8.2 Les bornes

Trois bornes encadrent un appel, et chacune répond à une panne différente.

- **Le délai maximum.** Chaque appel s'exécute sous un délai au-delà duquel il est annulé, et l'annulation devient un résultat d'erreur lisible par le modèle. Mon socle applique par défaut 30 secondes à chaque appel d'outil.
- **La politique de reprise.** Un échec réseau se rejoue avec un délai croissant et une part d'aléa, un échec de validation ne se rejoue pas. La règle qui décide : un appel se rejoue s'il est idempotent, et la partie 5.5 explique comment le rendre tel.
- **La troncature du résultat.** Traitée en partie 7, elle protège le contexte d'un outil devenu bavard après une mise à jour.

8.3 L'isolation des pannes

Un outil qui échoue laisse les autres du même tour aller au bout. Un tour qui échoue entièrement rend au modèle de quoi comprendre, et la boucle continue. Une boucle qui ne peut plus progresser sort par la porte prévue : un dernier appel au modèle avec les outils désactivés, qui force une réponse rédigée à partir de ce qui a été obtenu.

Cette sortie forcée est ce qui distingue un agent qui rend une réponse dégradée d'un agent qui rend une exception. Elle sert aussi quand le plafond d'itérations est atteint, ou quand le budget de l'exécution est dépassé sous une politique qui exige une réponse.

9. Effets de bord, état et reprise

Tant qu'un agent lit, une panne coûte un rejeu. Dès qu'il écrit, une panne peut coûter un rendez-vous fantôme chez un client. Cette partie traite du passage de l'un à l'autre, et c'est celle qui décide de la confiance qu'une entreprise accorde à son agent.

9.1 Trois régimes d'effet

Régime	Exemple sur le fil rouge	Ce qu'il exige
Lecture	chercher_client, lire_dossier	rien de particulier
Écriture réversible	creer_devis_brouillon	une trace, et un moyen d'annuler
Écriture irréversible	poser_rendez_vous, envoyer_message	clé d'idempotence, checkpoint, confirmation

La distinction utile porte sur ce que voit le client final. Un devis à l'état brouillon vit dans le logiciel de facturation et se supprime sans que personne à l'extérieur en sache rien. Un message parti ne revient pas. Un rendez-vous posé engage l'entreprise devant son client, et son annulation est un second événement visible.

9.2 Le checkpoint avant l'effet

Le mécanisme qui rend la reprise possible tient en une phrase : **l'état est persisté avant l'action, et l'action porte la marque de son état**. En pratique, l'orchestrateur enregistre le point d'exécution, l'intention et sa clé d'idempotence, puis exécute. Un processus qui redémarre relit ce point, constate que l'appel était engagé, interroge le service sur la clé, et découvre qu'il a abouti.

Sans checkpoint, la reprise doit choisir entre ré-exécuter, au risque du doublon, et abandonner, au risque de la relance jamais partie. Avec checkpoint, la question ne se pose plus. C'est ce dispositif qui porte le résultat déjà cité de l'agent de relance : depuis décembre 2025, zéro relance envoyée deux fois.

9.3 La confirmation humaine

Certains effets méritent une main humaine, et la conception consiste à choisir lesquels plutôt qu'à en faire une règle générale. Trois formes coexistent :

- **La validation avant effet.** L'agent prépare, une personne valide. Adapté aux montants, aux engagements et aux premiers mois de mise en service.
- **La fenêtre d'annulation.** L'effet part avec un délai pendant lequel il se retire. Adapté aux messages sortants.
- **Le seuil.** En dessous d'un montant ou d'une portée, l'agent agit seul, au-dessus il demande. C'est le réglage qui tient dans la durée, parce qu'il concentre l'attention humaine là où elle rapporte.

Sur l'agent vocal exploité de janvier à mai 2026, la contrainte de latence interdisait la validation avant effet : un interlocuteur au téléphone n'attend pas qu'un humain relise. Le rendez-vous se posait donc seul, sur des créneaux ouverts d'avance et bornés en durée, et la confirmation revenait à l'artisan par notification. La conception d'un outil d'écriture se fait avec le canal en tête.

9.4 Le mode simulation

Un catalogue gagne à savoir tourner sans effet. Chaque outil d'écriture accepte un indicateur qui journalise l'intention et rend un résultat plausible sans rien exécuter. Ce mode sert à trois moments : la recette avant mise en service, le jeu d'un incident sur des données réelles, et la démonstration devant le client. Il coûte quelques lignes par outil, et il évite la base de test qui diverge de la production.

10. Sécurité : la carte, puis le document

Cette partie donne la carte du terrain et les 3 réflexes qui ne se négocient pas. Le traitement complet fait l'objet d'un document séparé, et les mécanismes cités ici y sont détaillés avec leurs contre-mesures.

10.1 Le principe qui commande tout le reste

Le résultat d'un outil entre dans la conversation sous forme de texte, à l'endroit exact où entrent les messages de l'utilisateur. **Un modèle ne dispose d'aucun moyen fiable de distinguer une donnée d'une instruction.** Une page web récupérée, un document indexé, un champ de commentaire dans votre base : tout cela peut contenir des phrases écrites pour être lues par un agent.

Le scénario tient en 3 lignes. Un client dépose une pièce jointe qui contient, en petits caractères, une consigne adressée à l'agent. L'agent lit la pièce jointe par un outil, la consigne entre dans son contexte, et il exécute ce qu'elle demande avec les droits que vous lui avez donnés. On parle d'**injection indirecte**, et c'est la famille d'attaque propre aux agents.

Ce qui rend une injection exploitable est une combinaison de trois conditions : l'agent accède à des données sensibles, il consomme du contenu venu de l'extérieur, et il dispose d'un moyen de faire sortir de l'information. Retirer une des trois conditions désamorce la combinaison, et c'est souvent la troisième qui se retire le plus facilement.

10.2 Les surfaces, en une page

Surface	La question à poser
Injection indirecte	Quels outils rendent du contenu écrit par un tiers ?
Privilèges	Sous quelle identité chaque outil s'exécute-t-il, et sur quelles données ?
Validation	Que se passe-t-il si un argument est hostile plutôt que maladroit ?
Exfiltration	Quels outils peuvent faire sortir de l'information, et vers où ?
Isolation	Que peut atteindre un outil qui exécute du code fourni ?
Secrets	Une clé peut-elle se retrouver dans un prompt, un résultat ou un journal ?
Plafonds	Que coûte une boucle qui appelle un outil payant mille fois ?
Dépendances	Qui écrit les serveurs MCP branchés, et qui décide de leurs mises à jour ?
Audit	Six mois plus tard, sait-on qui a demandé quoi, et ce qui a été exécuté ?

10.3 Les 3 réflexes

Une identité par outil, au privilège minimum. `chercher_client` accède aux clients en lecture, `creer_devis_brouillon` crée des brouillons et rien d'autre. Un agent qui s'exécute

avec un compte d'administration hérite de tout ce que ce compte sait faire, et une injection réussie hérite avec lui.

La validation vit dans le code. Un schéma JSON garantit une forme, il n'autorise rien. Traitez chaque appel comme venant d'un appelant hostile : chemins de fichiers normalisés et confinés, requêtes préparées, URL sortantes restreintes à une liste connue, identifiants confrontés au périmètre de la conversation en cours. C'est du code défensif ordinaire, appliqué à un appelant qui écrit ses arguments à partir d'un texte que quelqu'un d'autre a rédigé.

Une confirmation humaine sur l'irréversible, tant que la mesure n'existe pas. Le seuil se relève ensuite, à partir des chiffres constatés. Il se relève beaucoup plus facilement qu'il ne se rétablit après un incident.

Le document dédié traite chacune de ces surfaces avec ses contre-mesures : filtrage et marquage du contenu non fiable, portée des jetons, isolation des exécutions, chaîne d'approvisionnement des serveurs MCP, journal d'audit et rétention. Cette partie en est le sommaire.

11. Tester et évaluer un catalogue d'outils

Un catalogue d'outils se teste sur deux plans qui n'ont rien à voir l'un avec l'autre. Le premier est du test logiciel ordinaire : la fonction fait-elle ce qu'elle annonce. Le second porte sur la sélection : le modèle appelle-t-il le bon outil, avec les bons arguments, au bon moment. Le premier est déterministe et gratuit, le second demande de vrais appels et une méthode.

11.1 Ce qui se teste hors ligne

Sans appeler le moindre modèle, on couvre déjà l'essentiel : l'exécution de chaque outil sur ses cas nominaux et ses cas limites, le rejet des arguments invalides par le schéma, la forme du résultat, la troncature au seuil, le message rendu sur chaque famille d'erreur, l'idempotence vérifiée en appelant deux fois avec la même clé, et le mode simulation qui ne produit aucun effet.

11.2 Ce qui se mesure en ligne

La sélection se mesure sur un jeu de cas de référence : une demande d'entrée, la séquence d'outils attendue, et le résultat attendu. Quatre indicateurs suffisent à piloter.

- **Le bon outil choisi.** Part des cas où la séquence appelée correspond à la séquence attendue.
- **Les arguments valides du premier coup.** Part des appels qui passent la validation sans correction.

- **Les appels inutiles.** Nombre d'appels au-delà de la séquence attendue, indicateur direct du coût et de la latence.
- **Les boucles.** Cas où le même outil est appelé plus de deux fois avec des arguments proches.

Le but n'est pas d'atteindre 100 %. Le but est de disposer d'un chiffre stable, qui bouge quand on touche une description ou qu'on change de modèle, et qui répond à la question « est-ce que ma modification a amélioré ou dégradé le routage ». Rejouer chaque cas 3 fois révèle une troisième catégorie, distincte du succès et de l'échec : le cas instable, qui passe deux fois sur trois. Il se corrige d'abord par la température et par la description, rarement par le code.

11.3 Le contre-exemple, encore

Les deux outils génériques de la partie 5 sont in-testables, et c'est un argument plus solide que tous les autres. **requete_sql(requete)** accepte un ensemble infini d'entrées, ce qui interdit d'écrire un jeu de cas de référence qui veuille dire quelque chose. Un outil qu'on ne peut pas mesurer est un outil dont personne ne saura dire, dans six mois, s'il s'est dégradé.

12. Observer et chiffrer

Un agent qui tourne se relit. La question n'est pas de savoir s'il a répondu, mais quel outil a été appelé, combien de temps il a pris, ce qu'il a rendu, et ce que cela a coûté aux tours suivants.

12.1 L'événement par appel

Chaque appel d'outil produit une ligne dans un flux d'événements, distinct des journaux applicatifs et conçu pour être chargé tel quel dans un tableur ou une base d'analyse. Les champs qui servent réellement : identifiant d'exécution et de session, nom de l'outil, verdict, latence, taille du résultat rendu, et le motif d'erreur quand il y en a un.

Trois lectures se font à partir de ce flux, et elles justifient à elles seules son existence. Quels outils sont réellement appelés, ce qui révèle les outils morts que le catalogue continue de payer à chaque tour. Quels outils échouent, et sur quel motif. Quels outils sont appelés deux fois de suite, ce qui pointe une description ambiguë.

12.2 Le coût d'un outil

Un outil coûte à deux endroits, et le second surprend. Le coût direct, quand l'outil est un modèle spécialisé ou un service facturé. Le coût induit, quand le résultat rendu s'ajoute au contexte de tous les tours suivants et se paie autant de fois qu'il reste dans la conversation.

Un ordre de grandeur, tiré du guide précédent de cette série : sur mon agent de relance, les 3 premières lignes de la facture sont celles de l'orchestrateur, et elles pèsent 76 % du total. L'orchestrateur est précisément le rôle qui relit chaque résultat d'outil à chaque tour. Réduire ce qu'un outil rend agit donc sur le poste principal de la facture, ce qui en fait le levier le plus rentable après la répartition des rôles entre modèles.

Deux plafonds encadrent le tout : un nombre maximum d'appels par exécution, et un plafond de dépense vérifié à chaque tour de boucle. Sur mon agent de relance, la boucle est bornée à 25 appels et le plafond réglé un tiers au-dessus du coût nominal d'une exécution.

13. Faire vivre le catalogue

Un catalogue vit plus longtemps que la version du modèle qui l'utilise, et il change pour des raisons qui viennent du métier plutôt que de la technique. Trois opérations reviennent, et chacune a un piège.

Renommer. Le nom d'un outil apparaît dans les conversations persistées, dans les jeux de cas de référence et dans les journaux. Un renommage se fait avec une période où les deux noms répondent, puis un retrait. Le nom n'a pas à être joli, il a à être stable.

Changer une description. C'est une modification de routage, et elle mérite le même traitement qu'une modification de code : passage du jeu de cas de référence avant et après, avec les 4 indicateurs de la partie 11. Une description améliorée pour un outil dégrade parfois la sélection d'un outil voisin, et c'est invisible sans mesure.

Scinder ou fusionner. Un outil dont les arguments se sont multipliés au fil des besoins demande à être scindé. Deux outils que le modèle confond systématiquement demandent à être fusionnés derrière une énumération. Les deux symptômes se lisent dans les mêmes indicateurs.

Une pratique évite la plus grande partie de ces reprises : versionner le catalogue avec la configuration, et l'épingler. Un agent en production doit pouvoir dire quelle version de son catalogue a produit une exécution donnée, au même titre qu'il dit quelle version de modèle l'a produite.

14. Ce que je retiens

- **Le périmètre d'action d'un agent est une liste écrite par un développeur.** Elle se lit, elle se relit avec le client, et elle se restreint.
- **Un outil se nomme comme une action métier.** Les outils génériques qui exposent une base ou une URL déplacent la conception vers le modèle, là où elle se mesure le moins bien.
- **La description est le routage.** C'est du prompt, facturé à chaque tour, et c'est le premier endroit à corriger quand un agent appelle mal.

- **Ce qu'un outil rend compte autant que ce qu'il fait.** Le résultat reste dans le contexte, et l'orchestrateur le relit à chaque tour.
- **Une erreur rendue au modèle se corrige au tour suivant.** Une exception qui remonte arrête l'exécution.
- **Un outil qui écrit porte une clé d'idempotence et un checkpoint.** C'est ce qui sépare un agent qui reprend d'un agent qui envoie deux fois.
- **Le résultat d'un outil est du contenu non fiable.** Le moindre privilège, la validation côté code et la confirmation sur l'irréversible sont les trois réflexes qui tiennent.
- **Un catalogue se mesure.** Bon outil choisi, arguments valides, appels inutiles, boucles : quatre chiffres qui rendent visible ce que le prompt seul laisse deviner.

15. À propos

Je conçois et j'intègre des agents IA destinés à tourner en production, pour des PME et des artisans. Vingt ans de développement, dont 7 ans de migration de données bancaires chez un cabinet de conseil, où un traitement doit s'exécuter une fois et une seule, reprendre après incident et se laisser auditer.

J'écris et je maintiens un socle d'orchestration public. Il porte les mécanismes décrits ici : validation des arguments avant appel, exécution parallèle bornée, résultats normalisés et tronqués, journal par exécution, plafonds de coût, reprise sur checkpoint. Il arrive écrit et testé sur un projet le premier jour, et ce qui reste à construire est le métier du client.

Le code est public : github.com/denislamard/loom

Le détail technique et les cas d'usage : denislamard.github.io

Annexe A. Lexique

Terme	Définition
Outil	Fonction déclarée à un modèle sous la forme d'un nom, d'une description et d'un schéma d'entrée, exécutée par votre code.
Catalogue	Ensemble des outils exposés à un agent pour un tour donné.
Schéma d'entrée	Description formelle des arguments attendus, en JSON Schema, servant à la validation avant exécution.
Function calling	Mécanisme par lequel un modèle demande l'appel d'un outil au lieu de rédiger une réponse.
tool_choice	Paramètre d'appel décidant si le modèle peut, doit ou ne peut pas demander un outil.
MCP	Model Context Protocol, protocole ouvert d'exposition d'outils par un serveur local ou distant.
Rôle-outil	Modèle spécialisé, avec son propre prompt système, exposé à l'orchestrateur comme un outil.
RAG	Recherche documentaire exposée comme outil, rendant des extraits de corpus au modèle.
Idempotence	Propriété d'une opération dont la répétition produit le même effet qu'une exécution unique.
Clé d'idempotence	Identifiant d'intention transmis au service appelé, lui permettant de reconnaître un rejeu.
Checkpoint	État persisté avant une action, permettant à une exécution interrompue de reprendre sans doublon.
Injection indirecte	Instruction placée dans un contenu que l'agent lit par un outil, dans l'intention d'orienter son comportement.
Moindre privilège	Principe d'attribution à chaque outil des seuls droits nécessaires à sa fonction.
Sortie forcée	Dernier appel au modèle avec les outils désactivés, garantissant une réponse rédigée.
Cas de référence	Jeu de cas d'entrée avec séquence d'outils attendue, servant à mesurer la qualité de sélection.

Annexe B. Fiche de mise en production d'un outil

Une page à parcourir avant d'exposer un outil à un agent qui touche des données réelles.

Conception

- Le nom désigne une action métier et reste stable dans le temps.
- La description dit ce que fait l'outil, quand l'appeler, ce qu'il rend, et ce qui le distingue de son voisin le plus proche.
- Le schéma compte 5 arguments au plus, avec des types fermés et une description par argument.
- La lecture et l'écriture sont portées par des outils distincts.

Exécution

- Un délai maximum est fixé, et son dépassement produit un message lisible par le modèle.
- La politique de reprise est cohérente avec le caractère idempotent de l'appel.
- Le résultat est tronqué à un seuil connu, et la troncature est annoncée dans le corps du résultat.
- Chaque famille d'erreur produit un message qui dit ce qui a échoué et ce qui reste possible.

Effets

- Le régime est identifié : lecture, écriture réversible, écriture irréversible.
- Un outil d'écriture porte une clé d'idempotence construite à partir de l'intention.
- Un checkpoint est enregistré avant l'action et relu à la reprise.
- Le mode simulation existe et ne produit aucun effet observable.

Sécurité

- L'outil s'exécute sous une identité dédiée, au privilège minimum.
- Les arguments sont validés côté code, en supposant un appelant hostile.
- Aucun secret ne transite par un prompt, un résultat ou un journal.
- Le contenu rendu par un tiers est identifié comme tel avant d'entrer dans le contexte.
- Un plafond d'appels et un plafond de dépense encadrent l'exécution.

Mesure

- Les tests hors ligne couvrent les cas nominaux, les cas limites et chaque famille d'erreur.
- Le jeu de cas de référence contient au moins un cas où cet outil doit être appelé, et un cas où il doit être ignoré.
- L'appel produit un événement portant verdict, latence et taille de résultat.
- Le catalogue est versionné avec la configuration, et l'exécution enregistre la version utilisée.

Annexe C. Anatomie d'un outil, en code

Cette annexe s'adresse à l'équipe technique. Le corps du document se lit sans elle. L'outil retenu est **poser_rendez_vous**, parce qu'il porte les 4 propriétés qui comptent : un schéma fermé, une validation côté code, une clé d'idempotence, et une erreur rendue au modèle.

La déclaration

```
{
  "name": "poser_rendez_vous",
  "description": "Pose une visite technique dans l'agenda de l'entreprise et
    l'annonce au client. À appeler une fois le créneau confirmé
    par le client. Pour proposer des créneaux, utiliser
    chercher_creneaux.",
  "input_schema": {
    "type": "object",
    "properties": {
      "client_id": {"type": "string",
        "description": "Identifiant rendu par chercher_client"},
      "debut": {"type": "string", "format": "date-time",
        "description": "Début du créneau, ISO 8601, fuseau Europe/Paris"},
      "duree_min": {"type": "integer", "minimum": 30, "maximum": 240},
      "motif": {"type": "string", "enum": ["diagnostic", "metre", "depannage"]}
    },
    "required": ["client_id", "debut", "duree_min", "motif"]
  }
}
```

Quatre arguments, deux types fermés, une unité dans le nom, et une description qui nomme l'outil voisin dont il faut le distinguer.

L'exécution

```
async def poser_rendez_vous(
    client_id: str, debut: str, duree_min: int, motif: str, *,
    ctx: Contexte,
) -> str:
    # 1. Périmètre : l'identifiant appartient-il à la conversation en cours ?
    if client_id not in ctx.clients_consultes:
        return ("Ce client n'a pas été consulté dans cet échange. "
            "Appeler chercher_client avant de poser un rendez-vous.")

    # 2. Validation métier, au-delà de ce que le schéma garantit
    creneau = parse_iso(debut)
    if creneau < ctx.maintenant:
        return f"Le créneau demandé est passé. Date reçue : {debut}."

    # 3. Clé d'idempotence, construite sur l'intention
    cle = sha256(f"{client_id}|{creneau:%Y-%m-%dT%H:%M}|{motif}")

    # 4. Checkpoint avant effet
    await ctx.checkpoint.enregistrer("poser_rendez_vous", cle=cle)

    # 5. Appel, sous l'identité dédiée à l'écriture d'agenda
    try:
        rdv = await ctx.agenda.creer(
            client_id, creneau, duree_min, motif, idempotency_key=cle,
        )
    except CreneauOccupe as e:
        return (f"Ce créneau vient d'être pris. Créneaux voisins libres : "
            f"{', '.join(e.alternatives)}")
```



```
except AgendaIndisponible:
    return "L'agenda est momentanément injoignable. Réessayer dans un instant."

# 6. Résultat court, stable, réutilisable au tour suivant
return (f"Rendez-vous {rdv.reference} confirmé le "
        f"{rdv.debut:%d/%m/%Y à %Hh%M} pour {duree_min} minutes.")
```

Six commentaires, six décisions. Le périmètre confronte un argument au contexte de la conversation, ce qu'aucun schéma ne sait faire. La clé d'idempotence est construite sur l'intention, ce qui rend un rejeu inoffensif. Le checkpoint précède l'effet. Les deux erreurs attrapées rendent des phrases exploitables, dont l'une propose la suite. Le résultat tient en une ligne et porte une référence que le modèle citera au client.

Le test hors ligne

```
async def test_rejeu_ne_double_pas(agenda_double, ctx):
    args = dict(client_id="C-1042", debut="2026-09-14T09:30",
                duree_min=60, motif="diagnostic")
    r1 = await poser_rendez_vous(**args, ctx=ctx)
    r2 = await poser_rendez_vous(**args, ctx=ctx)
    assert agenda_double.appels_effectifs == 1
    assert r1 == r2

async def test_client_hors_perimetre(ctx):
    r = await poser_rendez_vous(client_id="C-9999", debut="2026-09-14T09:30",
                                duree_min=60, motif="diagnostic", ctx=ctx)
    assert "chercher_client" in r
    assert ctx.agenda.appels_effectifs == 0
```

Ces deux tests s'exécutent sans clé d'API et sans réseau. Le premier vérifie la propriété qui empêche un doublon, le second vérifie qu'un identifiant inventé par le modèle ne produit aucun effet.

Annexe D. Relevé daté

Relevé effectué en Juin 2026. Cette annexe rassemble ce qui change vite. Le corps du document se lit sans elle et reste valable quand elle se périmé.

Formats d'appel d'outil

Format	Déclaration	Retour d'un appel
Anthropic	tools, chaque entrée portant name, description, input_schema	bloc de contenu de type tool_use, réponse attendue en tool_result
OpenAI	tools, chaque entrée portant type function et parameters	tool_calls dans le message, réponse attendue en message de rôle tool

La plupart des fournisseurs tiers et des serveurs d'inférence auto-hébergés exposent le format OpenAI. Une couche fournisseur qui absorbe les deux rend le reste du système indifférent au choix.

Model Context Protocol

Le protocole distingue 3 primitives : les outils, appelés par le modèle ; les ressources, exposées en lecture ; les invites, proposées à l'utilisateur. Les transports en usage sont l'entrée et la sortie standard pour un serveur local, et HTTP pour un serveur distant. La spécification évolue plusieurs fois par an et ses versions se négocient au démarrage de la session, ce qui rend l'épinglage de version utile côté client comme côté serveur.

Repères de dimensionnement

Grandeur	Repère constaté	À vérifier sur votre agent
Taille de catalogue confortable	5 à 15 outils	mesurer la sélection au-delà de 20
Arguments par outil	5 au plus	le taux d'appels mal formés monte au-delà
Délai maximum par appel	30 secondes par défaut	à réduire sur un canal conversationnel
Troncature d'un résultat	50 000 caractères	à réduire si l'orchestrateur relit souvent
Surcoût du français en tokens	15 à 30 %	à mesurer sur vos propres textes

Ces valeurs viennent de mes agents et des configurations que j'exploite. Elles servent de point de départ, et se remplacent par une mesure sur vos dossiers dès le cadrage.